Übungsblatt 4: Aktorik & Sicherheit

Mikrocomputertechnik 3 · Zeitdiagramme, PWM-Rechnungen und überwachte Abläufe

Musterlösung

Ziel und Arbeitsweise

Sie berechnen Hardware-PWM, lesen Zeitdiagramme und beurteilen, ob ein Watchdog fehlenden Anwendungsfortschritt erkennen kann. Alle sechs Pflichtaufgaben sind als Papieraufgaben ohne Platine oder Simulator lösbar.

Bearbeitungszeit: etwa 60–75 Minuten selbstständig. **Orientierungspunkte:** 32. Im Unterricht sind 55 Minuten angeleitete Arbeitsphasen vorgesehen: Aufgabe 1 (10 min), Aufgaben 2–3 (15 min), Aufgabe 4 (15 min), Aufgaben 5–6 (15 min). Besprechungen gehören zu den Vorlesungsphasen.

Gemeinsames Modell: Der PWM-Timer erhält $f_{\text{tim}} = 100$ MHz. PSC, Prescaler oder Vorteiler, teilt durch PSC + 1. CNT, Counter oder Zählwert, zählt aufwärts $0, \ldots, \text{ARR}$; ARR ist das Auto-Reload Register. Jeder Zustand dauert einen ganzen Tick. Eine Periode hat $N = \text{ARR} + 1$ Intervalle. Beim Active-High-Ausgang gilt HIGH genau für CNT < CCR; CCR ist das Capture/Compare Register. Der Pin hat ideal die Pegel 0 V und 3,3 V. Andere Zählmodi und Polaritäten werden ausdrücklich angegeben. PWM bedeutet Pulsweitenmodulation, WDT Watchdog Timer.

Geben Sie Rechenweg, Einheit und die Bedeutung des Ergebnisses an. Alle Werte sind Modellannahmen, keine Messungen oder universellen Registervorgaben.

Aufgabe 1: Takt, Intervalle und Endpunkte (6 Punkte)

1. Für `PSC = 99`, `ARR = 999`, `CCR = 250`: Berechnen Sie Zählfrequenz, PWM-Frequenz, HIGH-Zeit und aktiven Anteil. Markieren Sie HIGH und LOW in einer Periode mit Zeitachse. (2 P)
2. Jemand ersetzt ARR durch `1000`, lässt PSC und CCR jedoch unverändert. Bleiben Frequenz und Tastverhältnis genau gleich? Geben Sie die neuen Werte und die Ursache der Abweichung an. (2 P)
3. Ein separates Auf/ab-Modell mit `ARR = 4` durchläuft $0, 1, 2, 3, 4, 3, 2, 1$ und danach wieder 0. Vergleichen Sie seine Periodendauer bei 1 MHz Zähltakt mit dem Aufwärtszählen $0, 1, 2, 3, 4$. Darf hier überall $N = \text{ARR} + 1$ stehen? (2 P)

Musterlösung 1

1. $f_{\text{cnt}} = 100\,\text{MHz}/100 = 1\,\text{MHz}$, ein Tick $1\,\mu\text{s}$. $N = 1000$, $f_{\text{PWM}} = 1\,\text{MHz}/1000 = 1$ kHz. HIGH dauert $250\,\mu\text{s}$, LOW $750\,\mu\text{s}$: $D = 250/1000 = 25\,\%$. Auf der Zeitachse liegt HIGH in $[0, 250\,\mu\text{s})$, LOW in $[250, 1000\,\mu\text{s})$, danach beginnt die nächste Periode. (1 P für Takt/Frequenz, 1 P für Zeiten/Diagramm)
2. ARR 1000 bedeutet 1001 ganze Intervalle: $f_{\text{PWM}} = 10^6/1001 \approx 999{,}001$ Hz, $D = 250/1001 \approx 24{,}975\,\%$. HIGH bleibt $250\,\mu\text{s}$, die Periode wird $1001\,\mu\text{s}$ lang. Der Endzustand zählt mit. (je 1 P)
3. Auf/ab hat acht Intervalle, $T = 8\,\mu\text{s}$ und $f = 125$ kHz. Aufwärts hat fünf, $T = 5\,\mu\text{s}$ und $f = 200$ kHz. Die Formel $N = \text{ARR} + 1$ ist hier nur für das Aufwärtsmodell gültig; das Auf/ab-Modell verwendet $N = 2\,\text{ARR}$ bei ARR größer null. Die beiden Perioden stehen hier im Verhältnis $8 : 5$, nicht $2 : 1$. (je 1 P)

Aufgabe 2: Mittelwert, Effektivwert und Polarität (6 Punkte)

Eine ideale Spannungsquelle liefert das ungefilterte PWM-Rechteck 0/3,3 V mit HIGH-Anteil $D_H = 25\,\%$.

1. Bestimmen Sie den zeitlichen Mittelwert $\overline{V}$ und den Effektivwert V_{RMS}. RMS bedeutet Root Mean Square. (2 P)
2. Berechnen Sie die mittlere Leistung an $R = 1\,\text{k}\Omega$. Welche Leistung hätte eine tatsächlich konstante Spannung $\overline{V}$ an demselben Widerstand? Erklären Sie den Unterschied. (2 P)
3. Nun sei der Ausgang Active-Low bei unverändertem aktiven Anteil von 25 %. Bestimmen Sie HIGH-Anteil und Spannungsmittelwert. Begründen Sie, weshalb aktive Lastzeit und HIGH-Anteil getrennt werden müssen. (2 P)

Musterlösung 2

1. $\overline{V} = D_H V_H = 0{,}825$ V. $V_{\text{RMS}} = V_H\sqrt{D_H} = 1{,}65$ V. (je 1 P)
2. $P = V_{\text{RMS}}^2/R = 2{,}7225$ mW. Konstante 0,825 V ergäben $0{,}825^2/1000 = 0{,}680625$ mW. Der ungefilterte Pin besitzt weiterhin zwei Pegel. Für die ohmsche Leistung wird das mittlere Quadrat gebraucht, nicht das Quadrat des Mittelwerts. (1 P je richtigem Leistungsvergleich mit Begründung)
3. Die aktive Zeit liegt bei Active-Low auf LOW. Daher ist $D_H = 75\,\%$ und $\overline{V} = 2{,}475$ V. Derselbe aktive Anteil beschreibt bei anderer Polarität einen anderen Spannungsmittelwert. Daraus folgt keine universelle Aussage über Motorleistung oder wahrgenommene LED-Helligkeit. (1 P Werte, 1 P Begründung)

Diese Leistungsrechnung setzt eine ideale Quelle und einen ohmschen Widerstand voraus; sie ist keine Belastbarkeitsempfehlung für einen GPIO-Pin.

Aufgabe 3: Flanken und gepufferte Compare-Werte (5 Punkte)

1. Software-PWM plant $50\,\mu\text{s}$ HIGH, danach $50\,\mu\text{s}$ LOW. Die fallende Flanke verspätet sich um $20\,\mu\text{s}$. Die anschließende LOW-Wartezeit bleibt $50\,\mu\text{s}$. Berechnen Sie neue Periode, Frequenz und HIGH-Anteil. (2 P)
2. Bei einem einzelnen Puls verschieben sich beide Flanken gleich um $5\,\mu\text{s}$. Ändert sich seine Breite? Was ändert sich trotzdem? (1 P)

3. Im Timer aus Aufgabe 1, Teil 1 wird CCR bei CNT 500 von 250 auf 750 geändert. Der ungepufferte Vergleich prüft kontinuierlich CNT $<$ CCR. Zeichnen Sie die HIGH-Abschnitte der betroffenen Periode und berechnen Sie ihren gesamten HIGH-Anteil. Vergleichen Sie mit einer gepufferten Übernahme erst beim nächsten Periodenanfang. (2 P)

Musterlösung 3

1. $t_H = 70\,\mu\text{s}$, $t_L = 50\,\mu\text{s}$, $T' = 120\,\mu\text{s}$. $f' = 1/T' \approx 8{,}333$ kHz, $D'_H = 70/120 \approx 58{,}33\,\%$. Dies folgt aus dem angegebenen Wartezeitmodell, nicht aus jeder beliebigen Interruptverzögerung. (1 P Zeiten, 1 P Frequenz/Anteil)
2. $t_{\text{Ende}} - t_{\text{Anfang}}$ bleibt gleich, da beide Zeiten um denselben Betrag wachsen. Die Lage des Pulses verschiebt sich. (1 P)
3. Sofort wirksam: HIGH in $[0, 250)$ und $[500, 750)$, jeweils in Mikrosekunden; insgesamt 500 von 1000 Ticks, also 50 %. Der schon inaktive Ausgang wird bei CNT 500 wieder aktiv. Gepuffert: in dieser Periode nur $[0, 250)$, also 25 %; in der nächsten 750 Ticks, also 75 %. Das kontinuierliche Vergleichsmodell ist vorgegeben; reale Timer verlangen die Prüfung ihrer Update- und Preload-Regeln. (je 1 P)

Aufgabe 4: Servoformat und C-Rechenmodelle (5 Punkte)

Ein gewähltes Servoformat wiederholt Pulse alle 20 ms; 1,5 ms ist ein mögliches Kommando. Der Timer zählt mit 1 MHz.

1. Bestimmen Sie ARR, CCR und aktiven Anteil für diesen Puls. Wie lang wäre ein Puls mit demselben Anteil bei 1 kHz PWM? (2 P)
2. Die folgende reine Rechenfunktion rundet auf den nächsten Tick, bei genau halbem Tick nach oben. Welchen Wert liefert `percent_to_ccr(1, 49)`, und welchen tatsächlichen Anteil bedeutet er? Was liefert `percent_to_ccr(255, 999)`? (1 P)

```
uint32_t percent_to_ccr(uint32_t percent, uint16_t arr) {
    if (percent > 100u) percent = 100u;
    uint32_t n = (uint32_t)arr + 1u;
    return (percent * n + 50u) / 100u;
}
```

3. Eine Fading-Folge verwendet einen neuen Schritt alle 10 ms:

```
uint8_t fade_percent(uint64_t step) {
    uint32_t phase = (uint32_t)(step % 200u);
    return (uint8_t)(phase <= 100u ? phase : 200u - phase);
}
```

Welcher Wert entsteht bei `step = 125`? Wie lang ist ein voller Zyklus? Wie sollte eine verspätete Hauptschleife den aktuellen Schritt bestimmen, wenn alte Zwischenwerte übersprungen werden dürfen? Ändert eine verspätete Sollwertaktualisierung zwangsläufig die Hardware-PWM-Periode? (2 P)

Die Typen stammen aus `<stdint.h>`; `u` kennzeichnet unsigned-Konstanten. Beide Funktionen berechnen Werte ohne Gerätezugriff oder Wartefunktion.

Musterlösung 4

1. $N = 20000$, ARR 19999, CCR 1500, $D = 1500/20000 = 7{,}5\,\%$. Bei einer Millisekunde Periode wären 7,5 % nur 75 μs. Gleicher Anteil ist daher nicht dasselbe Servo-Kommando. Pulsformat und zulässige Wiederholrate sind gerätespezifisch. (je 1 P)
2. $(1 \cdot 50 + 50)/100 = 1$ in Integerarithmetik: ein von fünfzig Ticks, also 2 % statt gewünschtem 1 %. `255` wird auf `100` begrenzt; das Ergebnis ist `1000`. Hundert Prozent erfordern CCR N und eine passende Registerbreite; bei ARR 65535 wäre $N = 65536$ für einen 16-Bit-CCR nicht darstellbar. Die Rückgabe ist deshalb 32 Bit breit. (1 P für beide Fälle samt Quantisierung)
3. Phase 125 ergibt $200 - 125 = 75\,\%$. $200 \cdot 10$ ms ergeben zwei Sekunden. Der aktuelle Schritt folgt aus verstrichener Systemzeit seit einem Startpunkt geteilt durch die Schrittzeit, bei geeignetem Zeitbereich. Bei einem Systemtimer mit 10 MHz sind 10 ms genau 100000 Ticks. Die Folge enthält keine Busy-Wait-Schleife. Die PWM-Trägerperiode bleibt bei laufendem Hardware-Timer und unverändertem ARR gleich; der Sollwert kann länger auf seinem alten Wert bleiben. (1 P Wert/Zyklus, 1 P Ablauf)

Aufgabe 5: Watchdogfristen und Zeitfenster (5 Punkte)

1. Ein idealer Watchdog hat zwei Sekunden Timeout. Feeds erfolgen bei 0, 0,5 und 1 s; danach fehlen sie. Das Hauptprogramm hängt ab 1,2 s. Wann ist der Reset, und wie lang kann das letzte PWM-Muster bis dahin noch weiterlaufen, falls Timer und Takt aktiv bleiben? (2 P)
2. Für denselben fest eingestellten Zähler sei das nominale Timeout zwei Sekunden, der Watchdogtakt aber um bis zu 20 % schneller oder langsamer. Berechnen Sie frühestes und spätestes Timeout. Passt ein größter vorgesehener Feedabstand von 1,4 s mit 0,2 s zusätzlicher Verzögerung vor das früheste Timeout? (2 P)
3. Ein separates Fenster-Modell erlaubt Feeds nur für $0{,}5\,\text{s} < a < 2\,\text{s}$; a ist das Alter seit dem letzten Feed. Beurteilen Sie $a = 0{,}2$ s, 1 s und 2,1 s. Ist bei $a = 1$ s ein Feed ohne frischen Anwendungsfortschritt sinnvoll? (1 P)

Musterlösung 5

1. Der letzte Feed startet die Frist neu: Reset bei $1 + 2 = 3$ s. Vom Hänger bis dahin sind $3 - 1{,}2 = 1{,}8$ s. Ein Hänger setzt den Watchdog nicht selbst zurück. (je 1 P)
2. Bei schnellerem Takt $T_{\min} = 2/1{,}2 \approx 1{,}667$ s, bei langsamerem $T_{\max} = 2/0{,}8 = 2{,}5$ s. $1{,}4 + 0{,}2 = 1{,}6$ s liegt rechnerisch vor $T_{\min}$, mit etwa 0,067 s verbleibendem Abstand. Dies gilt für die vorgegebenen Grenzen, nicht als allgemeine Sicherheitsreserve. (je 1 P)
3. 0,2 s ist zu früh, 1 s zeitlich gültig, 2,1 s liegt nach dem abgelaufenen Timeout. Ohne frischen kritischen Fortschritt soll auch bei 1 s kein Feed erfolgen. Die strikten Fenstergrenzen gehören zu diesem Modell; echte Randbedingungen sind im Datenblatt zu prüfen. (1 P)

Aufgabe 6: Fortschritt, Reset und Aktorzustand (5 Punkte)

Eine Überwachungsrunde verlangt **neuen** Fortschritt bei Sensorerfassung, Regelberechnung und Ausgabe. Nach jedem erfolgreichen Feed werden die Nachweise für die nächste Runde zurückgesetzt. Die hier betrachteten Feedzeiten liegen im gültigen Fenster.

1. Ergänzen Sie für jede Runde „Feed ja/nein“ und begründen Sie. Ein altes gesetztes Flag zählt nicht als neue Aktivität. (2 P)

Runde	Neuer Sensorwert	Neue Berechnung	Neue Ausgabe	Feed?
A	ja	ja	ja	
B	ja	nein	ja	
C	altes Flag	altes Flag	altes Flag	

2. Eine Timer-ISR läuft weiter, während die Regelberechnung hängt. Weshalb würde blindes Füttern in dieser ISR den Fehler verdecken? Genügt allein das Verschieben des Feeds in die Hauptschleife? (1 P)
3. Beim Timeout startet die CPU neu. Nennen Sie zwei zu prüfende Randbedingungen, bevor daraus ein sicherer Motorzustand gefolgert werden darf. Erklären Sie außerdem, warum nach dem Neustart die Resetursache ausgewertet werden sollte. (2 P)

ISR bedeutet Interrupt Service Routine, CPU Central Processing Unit. Für die Resetdomänen des konkreten Geräts ist hier kein Verhalten vorgegeben.

Musterlösung 6

1. A: Feed ja, alle kritischen Schritte haben in der aktuellen Runde Fortschritt gemacht. B: nein, neue Regelberechnung fehlt. C: nein, frühere Nachweise gelten nicht für eine neue Runde. Nach A müssen sie gelöscht oder eindeutig einer neuen Runde zugeordnet werden. (1 P Entscheidungen, 1 P Umgang mit alten Nachweisen)
2. Laufende ISR-Aufrufe beweisen keine neue Regelberechnung. Ihr Feed verschiebt trotzdem das Timeout. Auch die Hauptschleife kann an einer fehlerhaften Stelle weiterlaufen: Entscheidend sind frische kritische Fortschrittsnachweise. (1 P)
3. Zu prüfen sind beispielsweise Resetdomäne und Resetpegel des Timer-/GPIO-Ausgangs sowie der Pegel des Treiber-Enable-Eingangs. Außerdem kann die externe Motorversorgung weiter anliegen. Abschalten, Bremsen und Halten sind verschiedene Zustände. Die Resetursache hilft, einen Watchdogneustart von anderen Resetgründen zu unterscheiden und wiederkehrende Fehler zu erkennen; derselbe Programmfehler kann nach dem Neustart erneut auftreten. Eine reine Feedüberwachung beweist auch keine richtige Berechnung. (1 P für zwei begründete Randbedingungen, 1 P Diagnose)

Optional: Rechenfunktionen am Host prüfen

Übernehmen Sie die beiden Funktionen aus Aufgabe 4 und die Funktion `may_feed` aus Vorlesung 4 in ein gewöhnliches C-Programm. Für `may_feed` benötigen Sie zusätzlich `<stdbool.h>`. Prüfen Sie beispielsweise:

- Prozentumsetzung: `0`, `25`, `100` und `255` bei ARR 999; außerdem `1` bei ARR 49 und `100` bei ARR 65535.
- Fading: Schritte 0, 100, 125, 199, 200; alle Ergebnisse sollen zwischen 0 und 100 liegen.
- Feedentscheidung: Alter 500, 501, 1999, 2000 ms, jeweils mit und ohne frischen Fortschritt.

Übersetzen Sie etwa mit `gcc -std=c11 -Wall -Wextra -Werror`. Die Hostausführung prüft Integerrechnung und Bedingungen, keine Pinflanken, Preload-Übernahme oder Resetdomänen. Ein Geräteversuch benötigt zuerst einen passenden dokumentierten Timer-/Watchdogtreiber sowie beobachtbare Ein- und Ausgänge; aus den Funktionsnamen entstehen keine Registeradressen.